

## 1. Arithmetic Operators

| Operator | Name                      | Function (Meaning)                                                                 | Example                                      |
|----------|---------------------------|------------------------------------------------------------------------------------|----------------------------------------------|
| +        | Addition / Unary Plus     | - Binary: Adds two numbers<br>- Unary: Represents positive value                   | $5 + 3 = 8$<br>$+a$                          |
| -        | Subtraction / Unary Minus | - Binary: Subtracts two numbers<br>- Unary: Negates a number                       | $5 - 3 = 2$<br>$-a$                          |
| *        | Multiplication            | Multiplies two numbers                                                             | $5 * 3 = 15$                                 |
| /        | Division                  | Divides numerator by denominator<br>Integer division if both operands are integers | $5 / 2 = 2$ (int)<br>$5.0 / 2 = 2.5$ (float) |
| %        | Modulus (Remainder)       | Gives remainder after division (only works with integers)                          | $5 \% 2 = 1$                                 |

### ◆ Basic Rules of Arithmetic Operators in C

- 1. Integer Arithmetic**
  - If both operands are integers, result is always an integer.
  - Example:  $5 / 2 = 2$
- 2. Mixed Arithmetic**
  - If one operand is float and other is int, result is float.
  - Example:  $5 / 2.0 = 2.5$
- 3. Modulus Operator %**
  - Works only with integers.
  - Result's sign is same as the first operand.
  - Example:  $5 \% 2 = 1$ ,  $-5 \% 2 = -1$
- 4. Unary Operators**
  - $+a$  → gives positive value (no change).
  - $-a$  → negates the value.
- 5. Division Rule**
  - $\text{int} / \text{int} = \text{int}$  (fraction part discarded).
  - $\text{int} / \text{float}$  OR  $\text{float} / \text{int}$  OR  $\text{float} / \text{float} \rightarrow$  result is float.
- 6. Precedence & Associativity**
  - $*, /, \%$  have higher precedence than  $+$  and  $-$ .
  - Operators of same precedence are evaluated left to right.
  - Example:  $5 + 2 * 3 = 11$  (not 21).

## Some programs to understand DIVISION(/) AND MODULAR DEVISION(%)

### 1. WAP to find out the sum of the digit of a 4 digit integer.

```
sumdigit.c
1 #include <stdio.h>
2 main()
3 {
4     int n,t,d1,d2,d3,d4,sum; //t=temporary variable to store actual INTRGER
5     printf("Enter a 4 digit Integer: ");
6     scanf("%4d",&n);
7     t=n;
8     d1=n%10;
9     n=n/10;
10    d2=n%10;
11    n=n/10;
12    d3=n%10;
13    n=n/10;
14    d4=n%10;
15    sum=d1+d2+d3+d4;
16    printf("The sum of all digit of %d= %d",t,sum);
17
18 }
```

C:\Users\USER\OneDrive\Desktop\practice c\sumdigitloop.exe

```
Enter a Integer no: 234256
The sum of all digit of 234256= 22
-----
Process exited after 6.311 seconds with return value 34
Press any key to continue . . .
```

### 2. WAP to find out the sum of the digit of a Integer number (Using LOOP).

```
sumdigitloop.c
1 #include <stdio.h>
2 main()
3 {
4     int t,n,d,sum=0;
5     printf("Enter a Integer no: ");
6     scanf("%d",&n);
7     t=n;
8
9     for (n=t;n>0;n=n/10)
10    {
11        d=n%10;
12        sum=sum+d;
13    }
14    printf("The sum of all digit of %d= %d",t,sum);
15 }
```

C:\Users\USER\OneDrive\Desktop\practice c\sumdigitloop.exe

```
Enter a Integer no: 234256
The sum of all digit of 234256= 22
-----
Process exited after 6.311 seconds with return value 34
Press any key to continue . . .
```

3. WAP to covert time duration in sec into hr:min:sec format.

```
1 #include <stdio.h>
2
3 main()
4 {
5     int sec, h, m, s;
6
7     printf("Enter time in seconds: ");
8     scanf("%d", &sec);
9
10    h = sec / 3600;    // 1 hour = 3600 seconds
11    m = (sec % 3600) / 60; // Remaining minutes
12    s = sec % 60;      // Remaining seconds
13
14    printf("%d seconds = %d hr : %d min : %d sec\n", sec, h, m, s);
15
16
17 }
18
```

C:\Users\USER\OneDrive\Desktop\practice c\timeduration.exe

Enter time in seconds: 25000  
25000 seconds = 6 hr : 56 min : 40 sec

4. WAP to reverse the digit of a 4 digit integer no.

```
1 #include <stdio.h>
2 main()
3 {
4     int n, rev = 0, d;
5
6     printf("Enter an integer number: ");
7     scanf("%d", &n);
8
9     // for loop runs as long as n > 0
10    for (; n > 0; n = n / 10) {
11        d = n % 10;    // extract last digit
12        rev = rev * 10 + d; // append digit to reversed number
13    }
14
15    printf("Reversed number = %d\n", rev);
16
17 }
```

C:\Users\USER\OneDrive\Desktop\practice c\revintloop.exe

Enter an integer number: 56789  
Reversed number = 98765

5. WAP to reverse the digit of a integer no (Using LOOP ).

```
1  #include <stdio.h>
2  main()
3  {
4      int n, d1, d2, d3, d4, rev;
5
6      printf("Enter a 4-digit number: ");
7      scanf("%d", &n);
8
9      d1 = n % 10;          // last digit
10     n = n / 10;
11
12     d2 = n % 10;          // 2nd digit
13     n = n / 10;
14
15     d3 = n % 10;          // 3rd digit
16     n = n / 10;
17
18     d4 = n % 10;          // 4th digit
19
20     rev = d1 * 1000 + d2 * 100 + d3 * 10 + d4;
21
22     printf("Reversed number = %d\n", rev);
23
24
25 }
```

C:\Users\USER\OneDrive\Desktop\practice c\reverseint.exe

Enter a 4-digit number: 5678  
Reversed number = 8765

-----